

TRANSFORMER

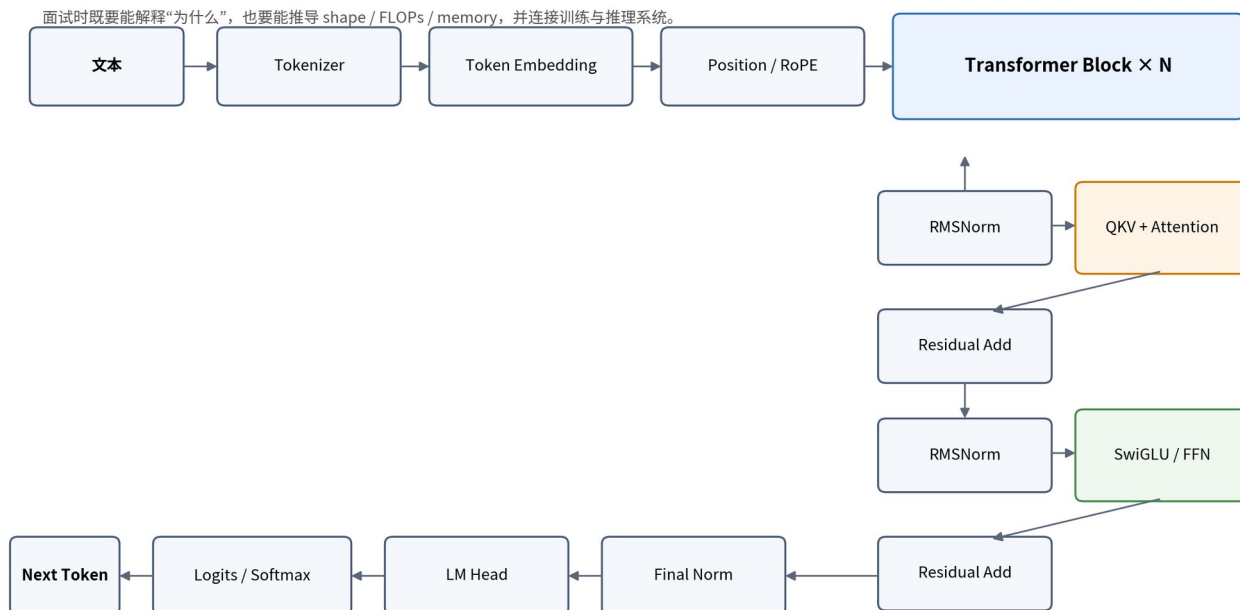
全方面面试宝典

从数学原理、架构演化到训练、分布式与 LLM Serving

2026 Edition · 面向 Google / OpenAI / DeepMind / Meta / 字节 / 阿里等算法与 AI Infra 岗位

Transformer / LLM 全链路知识地图

面试时既要能解释“为什么”，也要能推导 shape / FLOPs / memory，并连接训练与推理系统。



核心原则：不要只背“是什么”，要能回答“为什么、怎么推导、改了会怎样、系统代价是什么”。

如何使用这本宝典

建议的面试准备顺序

第一轮：只看每章“30 秒回答”和公式；第二轮：做白板推导；第三轮：刷 Gotchas 与系统题；第四轮：按题库模拟追问。
高阶面试的区分度主要来自 trade-off、边界条件和工程细节，而不是术语数量。

层级	面试官在测什么	你应该做到什么
L1 基础	概念是否准确	30 秒给出定义、输入输出和核心公式
L2 数学	是否真正理解	能从 shape、概率或线性代数推导关键结论
L3 训练	是否做过模型	能解释初始化、Norm、LR、loss spike、混合精度
L4 系统	是否懂大模型工程	能估算显存/FLOPs/KV cache，并比较并行与 serving 方案
L5 研究	是否能做设计判断	能提出 ablation、识别 confounder，并解释 trade-off

知识地图与核心公式

- Attention = 信息路由；FFN = 逐 token 的非线性计算；Residual Stream = 跨深度共享状态。
- 训练阶段最常见的难题：数值尺度、梯度流、数据/计算最优分配与分布式通信。
- 推理阶段最常见的难题：KV Cache、内存带宽、batching、prefill/decode 不同瓶颈。
- 现代 decoder-only LLM 的常见骨架：Tokenizer + Embedding + RoPE + Pre-RMSNorm + GQA/MLA + SwiGLU/MoE + LM Head。

Attention(Q,K,V) = softmax((QK^T + M) / sqrt(d_k)) V

KV_cache_bytes ≈ 2 × B × L × N_layers × N_kv_heads × d_head × bytes_per_element

Dense FFN params ≈ 2 d d_ff; SwiGLU params ≈ 3 d d_int

CrossEntropy(z,y) = logsumexp(z) - z_y; ∂L/∂z_i = p_i - 1[i=y]

目录

Part	主题
Part 1	Transformer 架构总览与演化
Part 2	Tokenizer / Embedding / Position
Part 3	Attention 全家桶
Part 4	Residual / Normalization / FFN
Part 5	MoE 与现代架构
Part 6	目标函数、训练稳定性与 Scaling
Part 7	后训练与参数高效微调
Part 8	分布式训练
Part 9	LLM 推理与 Serving

Part	主题
Part 10	长上下文、效率与量化
Part 11	Debug / 系统设计 / Research Sense
Part 12	终极面试题库与速查表

PART 1

Transformer 架构总览与演化

先建立“整机”视角，再进入每个零件。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

1.1 Transformer 为什么替代 RNN/Seq2Seq

30 秒回答

Transformer 的核心贡献不是“发明 attention”，而是把 sequence modeling 的主计算从递归状态更新改成全局自注意力 + 逐位置 FFN，从而让训练时所有 token 可并行，并把长距离依赖路径从 $O(L)$ 缩短到 $O(1)$ 个 attention hop。代价是标准 self-attention 的 $O(L^2)$ 计算/内存。

维度	RNN/LSTM	Transformer
训练并行	沿时间步递归，难并行	序列维可并行
长距离路径	依赖多个 recurrent steps	任意两 token 可一层直接交互
归纳偏置	强顺序递归	弱顺序偏置，需位置机制
长序列代价	时间线性但串行	attention 二次复杂度
扩展到大模型	通信/利用率不理想	GEMM 友好，适合 GPU/TP

Q. 为什么 Transformer 训练更容易并行，但自回归推理仍然串行？

A. 训练时目标 token 已知，可一次构造整段 causal mask；推理时第 $t+1$ 个 token 的输入取决于第 t 个采样结果，所以 token 级依赖仍然存在。

追问：继续追问 speculative decoding 如何利用并行验证减少串行步数。

Q. Self-attention 的最短依赖路径为什么是 $O(1)$ ？

A. 同一层中任意 query 都可直接与任意 key 计算相关性，因此信息可单跳交互；这不代表模型一定“学会”长距离依赖，只代表架构允许。

1.2 Encoder / Decoder / Decoder-only

架构	Attention 可见性	典型目标	典型使用
Encoder-only	双向 full attention	MLM / representation learning	BERT 类理解/检索/分类
Encoder-Decoder	Encoder 双向；Decoder causal；含 cross-attn	seq2seq / span corruption	T5、翻译、条件生成
Decoder-only	causal self-attention	next-token prediction	GPT/LLaMA 类通用生成

Autoregressive LM: $p(x_1, \dots, x_T) = \prod_t p(x_t | x_{<t})$

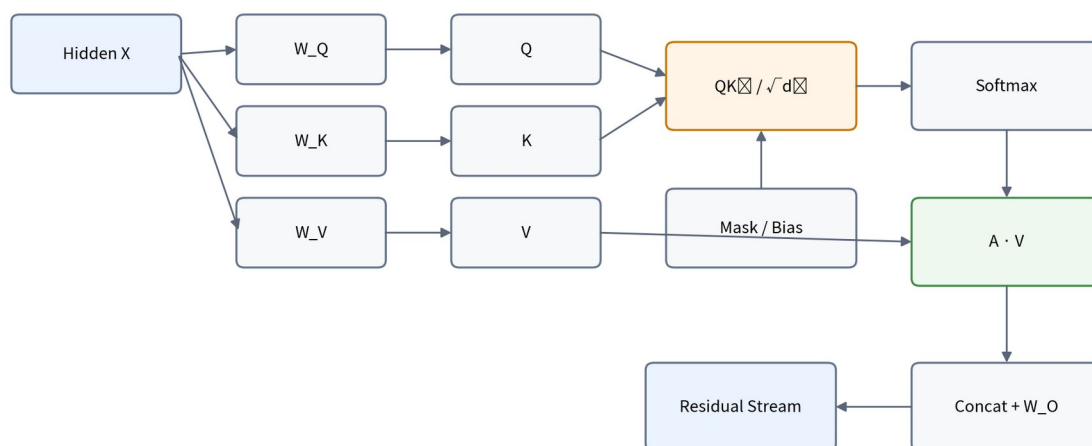
Q. 为什么 decoder-only 能做“理解”任务？

A. next-token objective 迫使模型压缩前缀中的语义、事实和关系来预测后续；通过 prompt 把分类/抽取改写成条件生成即可利用这些表示。它不是双向 encoder，但并不意味着没有理解能力。

Q. Cross-attention 的 Q/K/V 分别来自哪里？

A. Q 来自 decoder 当前隐状态；K/V 来自 encoder 输出。直觉上 decoder 提问“我现在需要什么”，encoder memory 提供“源序列有什么”。

1.3 一层 Decoder Block 的数据流

Attention：从“匹配”到“信息路由”

Q/K 共同决定 WHERE to read；V 决定 WHAT to read；W_O 决定多头结果如何重新写回 residual stream。

```

h = token_embedding(ids)
for block in blocks:
    h = h + attention(rmsnorm(h), position=rope)
    h = h + swiglu(rmsnorm(h))
logits = lm_head(final_norm(h))
  
```

- 现代 LLM 常用 Pre-Norm：Norm 放在 sublayer 输入侧，identity residual path 更直接。
- RoPE 通常作用于 Q/K 而非 V，因为位置主要影响“匹配谁”，而非“被取出的内容坐标”。
- GQA/MQA/MLA 主要优化 KV Cache 与 decode 带宽，不改变“query 读取 history”的基本语义。

1.4 Shape / 参数量 / FLOPs 白板题

对象	Shape (batch B, length L, hidden d)	关键点
Hidden	[B,L,d]	residual stream 主状态
Q	[B,Hq,L,dh]	$Hq \times dh = d$ (常见)
K/V	[B,Hkv,L,dh]	GQA 时 $Hkv < Hq$
Attention logits	[B,Hq,L,L]	naive 实现会物化 L^2
FFN hidden	[B,L,d_ff] 或 d_int	逐 token 计算
LM logits	[B,L,V]	大词表可能很重

Approx block params (MHA + dense FFN) $\approx 4d^2 + 2d \cdot d_{ff}$ (ignore bias/norm)

If $d_{ff} = 4d$, block params $\approx 12d^2$; attention projections $\approx 4d^2$, FFN $\approx 8d^2$

Q. 为什么很多 Transformer 参数主要在 FFN，而不是 attention matrix?

A. attention matrix QK^T 是激活而不是参数；参数主要来自 Q/K/V/O 四个 $d \times d$ 投影，以及 FFN 的扩维/降维矩阵。经典 $d_{ff}=4d$ 时 FFN 约 $8d^2$ ，约为 attention 投影的两倍。

PART 2

Tokenizer / Embedding / Position

输入表示决定 token 预算、序列长度和模型看到的坐标系。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

2.1 Tokenizer：不是“前处理”，而是模型接口 ABI

30 秒回答

Tokenizer 把 raw text 映射为有限词表 ID。现代 LLM 多用 BPE/Unigram/byte-level 变体。选择 tokenizer 会改变 sequence length、跨语言 fertility、代码/空白表示、special-token 协议以及 embedding/LM-head 参数量，因此必须和模型、数据、Serving 一起设计。

方法	训练思想	编码特性	典型 Gotcha
BPE	反复合并高频 pair	确定性 greedy merge	频率驱动，不直接最大化概率
WordPiece	基于似然/评分选择子词	常见 longest-match	实现细节与 BPE 易混
Unigram LM	从大词表删 token，最大化分词似然	可 Viterbi / sampling	需要概率模型
SentencePiece	raw text 上训练，统一空格处理	适合多语言	normalization 可能改语义
Byte-level BPE	bytes 保证覆盖	几乎无 UNK	可读性下降、序列长度 trade-off

Q. 为什么“token 越少越好”不是 tokenizer 的充分目标？

A. 压缩率降低序列长度，但过大词表会增加 embedding/LM-head 参数和输出计算；罕见大 token 还可能训练不足。真正目标是训练效用、语言覆盖、序列成本与词表成本的联合优化。

Q. 换 tokenizer 为什么通常需要继续预训练？

A. token ID 到 embedding row 的语义绑定发生变化，原模型看到的序列边界和位置距离也变了；即使初始化新 embedding，也需要让整个 residual stack 适应新的 token distribution。

- Unicode normalization 可能把视觉等价字符、全角/半角、组合字符改写，代码/标识符任务尤其敏感。
- Special tokens 不只是词表条目，还与 chat template、loss masking、tool calling 协议绑定。
- Prompt 的字符串后缀若落在 tokenizer 边界中，追加文本后可能重新分词；不要假设 tokenization 是 prefix-stable。

2.2 Token Embedding：lookup table 与梯度稀疏性

$$\mathbf{E} \in \mathbb{R}^{(V \times d)}, \quad \mathbf{x}_t = \mathbf{E}[\text{token_id}_t] \quad (\text{equiv. } \text{one_hot}(\text{token}) \cdot \mathbf{E})$$

Know-why

Embedding lookup 的本质是选择矩阵的一行。一个 batch 中只有出现过的 token rows 获得直接 input-side gradient；因此 Zipf 长尾会造成 rare token gradient starvation。若与 LM Head weight tying，所有 token row 还会从 output-side softmax 获得梯度。

Q. Weight tying 为什么常见？

A. 输入 embedding $\mathbf{E}$ 与输出 unembedding $\mathbf{W}_U$ 使用同一矩阵，可显著减少 $V \times d$ 参数，并让输入/输出词空间共享几何。但两个角色的梯度统计并不对称，量化或扩词表时也会增加耦合。

Q. Vocabulary 增大对模型成本有什么相反影响？

A. 一方面 tokenization 更细致/更短，可减少 L ，从而降低 attention 与主干 token FLOPs；另一方面 embedding/LM head 参数按 $V \times d$ 增长，输出 logits 计算也按 V 增长。

2.3 Position：绝对、相对、Bias、RoPE

家族	改在哪里	核心直觉	外推
Absolute PE	加到 hidden	给 token 一个绝对坐标	learned absolute 通常差

家族	改在哪里	核心直觉	外推
Relative representation	attention pair	直接建模 i-j	通常更自然
T5 relative bias / ALiBi	attention logits	按距离加偏置	ALiBi 有外推倾向
RoPE	旋转 Q/K	点积自然含相对相位	需处理 OOD frequency

RoPE property: $\langle R_m q, R_n k \rangle = \langle q, R_{(n-m)} k \rangle$

Q. 为什么 self-attention 没有位置编码时无法区分顺序？

A. 不考虑 mask 时，attention 对 token permutation 是 equivariant：输入位置交换只会对应交换输出。内容相似度本身不提供“第几个”的信息。

Q. 为什么 RoPE 通常只旋转 Q/K？

A. 位置需要改变 attention score 的相对匹配；V 是被聚合的内容。如果旋转 V，会把绝对位置相位混入被搬运的内容坐标，通常没有必要。

Q. 把 max_position_embeddings 改大为什么不等于获得长上下文？

A. 模型没有在新的位置相位/距离分布上训练，RoPE 频率进入 OOD 区域；同时数据 curriculum、attention pattern 和“有效检索能力”都可能限制真实长上下文。

PART 3

Attention 全家桶

QKV、Scaled Dot-Product、MHA/GQA/MLA、FlashAttention 与 KV Cache。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

3.1 Q / K / V：三种投影的真正职责

$$Q=XW_Q, K=XW_K, V=XW_V$$

$$q_i^T k_j = x_i^T W_Q W_K^T x_j$$

核心理解

Q/K 联合学习一个 bilinear matching space，决定 WHERE to read；V 决定 WHAT to read。把 Q/K/V 仅记成“查询/键/值”不够，高阶面试会继续追问它们的几何、梯度、范数和系统代价。

Q. 为什么不直接用 X 做 Q=K=V?

A. 分离投影允许“匹配特征”和“被传输内容”使用不同子空间；同一输入可以用某些维度决定相关性，再搬运另一组特征。多头又允许多组并行匹配空间。

Q. QK-Norm 解决什么问题?

A. 控制 Q/K 范数，避免 logits 仅因向量 norm 增长而极端放大，从而缓解 softmax saturation 和训练不稳定。它改变的是 attention geometry，不等同于 residual RMSNorm。

3.2 Scaled Dot-Product Attention

$$A = \text{softmax}((QK^T + M) / \sqrt{d_k}); O = A V$$

Q. 为什么除以 $\sqrt{d_k}$?

A. 若 q、k 各维近似独立、零均值、单位方差，则 $q \cdot k$ 的方差约为 d_k 。除以 $\sqrt{d_k}$ 把 logits 标准差拉回 $O(1)$ ，减少 softmax 饱和。

Q. Causal mask 为什么加 $-\infty$ 而不是把 softmax 后的概率乘 0?

A. 在 softmax 前屏蔽可保证被屏蔽位置不进入分母并得到精确 0 概率；softmax 后再乘 0 会破坏概率归一化，除非再重新归一化。

3.3 Stable Softmax / LogSumExp / Online Softmax

$$\text{softmax}(z_i) = \exp(z_i - m) / \sum_j \exp(z_j - m), m = \max_j z_j$$

$$\text{logsumexp}(z) = m + \log \sum_j \exp(z_j - m)$$

- 平移 logits 不改变 softmax；减最大值是数值稳定的核心。
- Online softmax 可按 block 维护 running max 与 running denominator，并在 max 更新时重标定旧 block 的贡献。
- FlashAttention 利用这种可分块的精确重标定，避免在 HBM 中物化完整 $L \times L$ attention matrix。

Q. FlashAttention 是近似 attention 吗?

A. 不是。核心版本计算与标准 softmax attention 数学等价的结果，只改变计算顺序和内存访问；其主要优化对象是 HBM↔SRAM IO，而非 attention 公式。

3.4 MHA → MQA → GQA → MLA

机制	Q heads	KV heads / cache	主要收益	主要代价
MHA	H	H	表达最自由	KV cache 最大
MQA	H	1	decode 带宽/显存最省	可能损失质量
GQA	H	G, $1 < G < H$	质量-速度折中	需要 group 映射
MLA	H	latent KV compression	进一步压缩 cache	结构/实现更复杂

$$\text{KV bytes/token} \approx 2 \times N_{\text{layers}} \times N_{\text{kv_heads}} \times d_{\text{head}} \times \text{bytes}$$

Q. 为什么 MQA/GQA 对训练 FLOPs 的收益不如对 decode 的收益明显？

A. 训练/prefill 常是大 GEMM 与 attention 计算主导；decode 时每步只处理少量新 token，却要读取长历史 K/V，成为 memory-bandwidth bound。减少 KV heads 直接减少每步历史读取量。

Q. GQA 的 group 数如何选择？

A. 没有脱离模型/硬件的唯一最优。更少 KV heads 节省 cache/带宽，但增加共享；需用 held-out quality、decode throughput、KV memory 和 TP mapping 联合 ablation。

Q. MLA 与 GQA 的本质区别？

A. GQA 仍缓存若干显式 K/V heads；MLA 对历史表示做低秩联合压缩，缓存 latent representation，并在计算 attention 时通过结构化投影恢复所需信息。

3.5 Attention 复杂度与真实系统瓶颈

阶段	主要 shape	常见瓶颈	优化
训练/prefill	$QK^T: L \times L$	compute + attention IO	FlashAttention, CP, sequence packing
decode	$1 \times L$ attention	KV cache read bandwidth	GQA/MQA/MLA, KV quantization
长上下文	L^2 激活/计算	显存 + time	window/sparse/CP/FlashAttention
大 batch serving	大量动态 KV	fragmentation / scheduling	PagedAttention, continuous batching

PART 4

Residual / Normalization / FFN

Transformer 能堆到几十/上百层，关键不只在 attention。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

4.1 Residual Stream: 跨深度共享状态总线

$$h_{l+1} = h_l + F_l(h_l)$$

30 秒回答

Residual connection 提供 identity path，使每层学习“增量更新”而不是重新构造完整表示；梯度也拥有绕过复杂分支的直接路径。现代 mechanistic 视角把 residual stream 看成所有 Attention/MLP 模块共享读写的状态总线。

Q. 为什么“残差解决梯度消失”这个说法不够完整？

A. identity Jacobian 确实改善梯度路径，但深层训练还受 residual branch scale、normalization placement、初始化、optimizer update 与激活方差影响。Residual 是条件之一，不是万能解释。

Q. Pre-Norm 为什么更容易训练？

A. $h + F(\text{Norm}(h))$ 保留未经 Norm/非线性变换的 identity path，梯度可直接沿 residual stream 传播；Post-Norm 的每层输出都会经过 Norm，初始化附近梯度分布更敏感。

- Pre-Norm 的代价之一是 residual stream magnitude 可能随深度增长，后层 update/stream ratio 变小。
- DeepNorm、ReZero、LayerScale 等都在重新控制 residual branch 的写入尺度。
- 分布式 fused dropout-add-norm 的算子顺序一旦错，往往不会立刻报错，但会改变模型函数。

4.2 LayerNorm vs RMSNorm

$$\text{LayerNorm}(x) = \gamma \odot (x - \mu) / \sqrt{\sigma^2 + \epsilon} + \beta$$

$$\text{RMSNorm}(x) = \gamma \odot x / \sqrt{\text{mean}(x^2) + \epsilon}$$

Q. RMSNorm 为什么在现代 LLM 常见？

A. 它去掉 re-centering，只做尺度归一化，结构更简单；实践上与 Pre-Norm residual 架构配合良好。真实 kernel 加速取决于 fused implementation 和 memory traffic，而不能只按“少算一次均值”估算。

Q. ϵ 只是为了防除零吗？

A. 不只是。 ϵ 决定小范数区域的归一化增益上界，因此会影响数值稳定、低精度行为和梯度放大。

Q. 为什么 QK-Norm 与 RMSNorm 不能混为一谈？

A. RMSNorm 通常控制 residual/branch 输入尺度；QK-Norm 直接归一化 query/key，限制 attention logit 的几何尺度。作用位置和优化目标不同。

4.3 FFN / MLP: 为什么是参数主体

$$\text{FFN}(x) = W_2 \varphi(W_1 x)$$

$$\text{SwiGLU}(x) = W_{\text{down}} [\text{SiLU}(W_{\text{gate}} x) \odot (W_{\text{up}} x)]$$

Q. 为什么 FFN 是 position-wise，却仍然非常重要？

A. attention 负责跨 token 混合，而 FFN 在每个 token 的 channel 维做高容量非线性变换；每层重复“通信→局部计算”，模型才能既聚合上下文又形成复杂特征。

Q. 为什么 SwiGLU 的 d_{int} 不能直接沿用 4d？

A. 普通两矩阵 FFN 参数约 $2d \cdot d_{\text{ff}}$ ；SwiGLU 有 gate/up/down 三矩阵，约 $3d \cdot d_{\text{int}}$ 。参数匹配经典 4d FFN 时， $d_{\text{int}} \approx 8d/3 \approx 2.67d$ ，再按硬件友好粒度取整。

Q. GELU 与 SiLU 的主要差异是什么？

A. 两者都是平滑非线性； $\text{SiLU} = x \cdot \text{sigmoid}(x)$ ，GLU/SwiGLU 用它作为 gate 分支并与另一 value 分支逐元素相乘，因此不仅是“换激活函数”，而是改变网络拓扑。

PART 5

MoE 与现代架构

把 FFN 从“所有 token 共用参数”改成“条件激活专家”。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

5.1 Sparse Mixture-of-Experts 基本结构

$$y = \sum_{e \in \text{TopK}(\text{router}(x))} p_e(x) \cdot \text{Expert}_e(x)$$

30 秒回答

MoE 通常把 Transformer 的 dense FFN 替换成多个 expert FFN，并由 router 对每个 token 只激活 Top-K experts。这样总参数可以大幅增加，而每 token 激活 FLOPs 只增加有限。真正难点转移到路由负载、all-to-all 通信、capacity 与训练稳定性。

概念	含义	常见风险
Router	token→expert scores	早期 collapse / 偏置
Top-K	每 token 激活专家数	K 越大 compute/communication 越高
Capacity factor	每 expert 可接收 token 上限	过小会丢 token/溢出
Load balance	避免专家冷热不均	aux loss 可能干扰主目标
Expert Parallel	专家分布到不同 GPU	all-to-all 成为瓶颈

Q. 为什么 MoE “参数多但 FLOPs 不同比例增加”？

A. 因为每个 token 只通过少数专家，而非所有专家。总参数反映模型容量，activated parameters 才更接近每 token 计算量。

Q. MoE 为什么常替换 FFN 而不是 attention？

A. FFN 本身是逐 token 独立计算，更自然做 token-level routing；attention 涉及 token 间交互和 KV cache，稀疏路由会引入更复杂的通信/缓存语义。

Q. Expert load imbalance 会造成什么系统问题？

A. 最慢/最热 expert 决定同步 step 时间，其他 GPU 空等；同时 token dropping/overflow 会影响模型质量。因此 router quality 和硬件拓扑高度耦合。

5.2 Switch / DeepSeekMoE / 现代 MoE 面试点

- Switch Transformer 将 routing 简化为 Top-1，强调通信与稳定性。
- 现代 MoE 常引入 shared experts + routed experts，让通用知识有稳定路径，同时让 routed experts 专门化。
- DeepSeek-V3 报告采用 auxiliary-loss-free load balancing，并结合 MLA 与 multi-token prediction；面试时重点不是背配置，而是解释为何要减少 auxiliary objective 对主任务的干扰。

PART 6

目标函数、训练稳定性与 Scaling

能训练出来，比“纸面架构漂亮”更重要。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

6.1 Causal LM / MLM / Span Corruption

目标	被预测位置	上下文可见性	代表
Causal LM	每个下一个 token	只看左侧	GPT/LLaMA
MLM	被 mask token	双向	BERT
Span corruption	被替换 span	encoder 双向 + decoder causal	T5

Q. Causal LM 的 labels 为什么要 shift?

A. 位置 t 的 hidden 表示基于 $x_{\leq t}$, 用来预测 x_{t+1} 。实现中常把 `logits[:-1]` 与 `labels[1:]` 对齐; 特殊 token、padding、packed sequence 边界需要正确 mask。

Q. Packing 为什么能提高训练效率?

A. 把多条短样本拼入固定 context, 减少 padding 浪费, 提高有效 token/step。但必须处理 document boundary、attention mask、loss mask 和 position IDs, 否则会引入跨样本泄漏。

6.2 Cross-Entropy、z-loss 与 logit 稳定

$$L = -\log \text{softmax}(z)_y = \log \text{sumexp}(z) - z_y$$

$$\partial L / \partial z_i = p_i - 1[i=y]$$

- Cross-entropy 对 logits 的整体平移不敏感, 因此 logit mean 可漂移; z-loss 可显式惩罚 log-partition。
- 大 logits 会增加低精度 overflow/underflow 风险, 也可能让 softmax 过于尖锐。
- loss 数值正常不代表所有 token 正常; 建议按 token type、domain、position 监控 NLL。

6.3 AdamW / LR / Warmup / Gradient Clipping

$$\text{AdamW: } \theta \leftarrow \theta - \eta \cdot m_hat / (\sqrt{v_hat} + \epsilon) - \eta \cdot \lambda \cdot \theta$$

Q. AdamW 为什么把 weight decay 与梯度更新解耦?

A. L2 regularization 在 adaptive optimizer 中会被每参数的 v 缩放, 效果不再等价于统一权重衰减; AdamW 把 decay 作为直接参数缩放项。

Q. Warmup 为什么在 Transformer 常见?

A. 训练初始统计量、Adam moments、激活与梯度尺度尚未稳定, 直接使用峰值 LR 容易产生大 update。Warmup 是稳定化手段, 但最佳长度与 Norm/初始化/批量大小耦合。

Q. Gradient clipping 是解决爆炸的万能方法吗?

A. 不是。它只限制最终梯度 norm, 可能掩盖 loss scale、attention logit、数据异常或优化器配置的根本原因; 过度 clipping 还会长期改变有效学习率和方向。

6.4 初始化与深度稳定

- 目标不是让每个矩阵“看起来方差合理”, 而是让 residual update、attention logits 和 optimizer update 随深度保持可控。
- Pre-Norm、residual scaling、output projection scaling、zero-init/gating 等方法都在调节“每层写多少”。
- 训练前应跑短程 scale probe: 记录各层 RMS、attention entropy、gradient norm、update/weight ratio, 而非只看第一步 loss。

6.5 Scaling Laws 与 Chinchilla 直觉

核心思想

模型性能随参数、数据和计算量呈经验幂律。给定固定 compute，模型参数和训练 token 应联合优化；Chinchilla 的重要结论是很多早期大模型相对“参数过大、数据不足”，增加训练 token 可以获得更好的 compute-optimal 性能。

Q. Scaling law 是否说明架构不重要？

A. 不是。幂律是在某一架构家族/训练 recipe 范围内的经验规律；更好的数据、tokenizer、optimizer 或架构会改变常数项甚至可用 scaling region。

Q. 为什么数据质量与数据量存在张力？

A. 强过滤提高平均 token 质量，但减少 unique tokens，长周期训练可能需要重复更多 epoch；最佳方案依赖训练 token horizon，并可通过数据 mixture/curriculum 实验决定。

PART 7

后训练与参数高效微调

从“会预测文本”到“会按指令行动”。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

7.1 SFT：监督信号到底加在哪里

30 秒回答

SFT 使用 instruction-response 或多轮对话数据继续训练。常见做法只对 assistant response token 计算 loss，prompt/user token 作为条件上下文但 loss mask=0。核心风险是模板错误、EOS/turn boundary 错误、数据风格单一和 teacher error 直接变成监督信号。

Q. 为什么 SFT 常做 response-only loss?

A. 目标是学习在给定 prompt 下生成 assistant 行为，不希望模型把 user 消息本身当作需要模仿的输出。某些预训练式 chat 数据也可对全序列算 loss，但语义不同。

7.2 LoRA / QLoRA

$$W_{\text{new}} = W_0 + \Delta W, \quad \Delta W = B A, \quad \text{rank } r \ll \min(d_{\text{in}}, d_{\text{out}})$$

Q. LoRA 为什么参数高效?

A. 冻结 W_0 ，只训练低秩 BA，训练参数和 optimizer state 从 $O(d^2)$ 降到 $O(r(d_{\text{in}}+d_{\text{out}}))$ 。推理时可 merge 回原权重或保留 adapter。

Q. LoRA rank 越大越好吗?

A. 不是。更大 r 增加容量与显存/计算，但数据不足时可能过拟合；不同模块的 intrinsic adaptation rank 也不同。应对 target modules、rank、alpha、dropout 做联合 ablation。

Q. QLoRA 的核心是什么?

A. 把冻结基座量化到 4-bit 以省显存，同时对 LoRA adapter 以较高精度训练并让梯度穿过量化基座。其关键不是“4-bit 也直接训练全部权重”。

7.3 DPO / RLHF 面试最小框架

- RLHF 典型：SFT → preference/reward model → policy optimization；核心难点是 reward misspecification 与 distribution shift。
- DPO 直接用 chosen/rejected pair 优化相对 preference，不显式训练 reward model/在线 RL，工程更简单。
- 面试时优先讲数据语义与优化目标，而不是把所有 post-training 算法混成“对齐”。

PART 8

分布式训练

显存、计算与通信：三者必须一起算。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

8.1 Data Parallel / DDP / FSDP / ZeRO

方法	每卡参数	梯度	Optimizer state	主要通信
DDP	完整	完整	完整	all-reduce gradients
ZeRO-1	完整	完整	分片	optimizer sync
ZeRO-2	完整	分片	分片	grad reduce-scatter
ZeRO-3 / FSDP	分片	分片	分片	param all-gather + reduce-scatter

Q. 为什么 ZeRO-3 省显存但通信更复杂？

A. 参数本身分片，计算某层前需要把该层参数 all-gather 到参与 rank；反向后再 reduce-scatter 梯度。显存换成了更频繁的通信与调度。

8.2 Tensor Parallel

核心

TP 在单个大矩阵内部切分，解决“单层矩阵放不下一卡/单卡 GEMM 太大”的问题；Transformer 常使用 column-parallel + row-parallel 配对，让中间激活避免不必要 all-gather。

- Attention QKV 常做 column parallel；output projection 做 row parallel。
- FFN gate/up 常 column parallel；down projection row parallel。
- GQA 时 KV head 数可能小于 TP world size，需要复制或采用特殊 sharding，容易出现 checkpoint/layout bug。

Q. TP 为什么通常更适合节点内 NVLink，而 DP 可跨节点？

A. TP 每层都有高频 collective，通信处在关键路径；DP 通常每 step 对梯度聚合，可更好 overlap 且频率更低，因此对网络延迟更不敏感。

8.3 Pipeline / Sequence / Context / Expert Parallel

并行	切什么	主要问题
Pipeline Parallel	按层切模型	pipeline bubble, stage imbalance
Sequence Parallel	部分激活/Norm 按序列切	需要配合 TP collective
Context Parallel	长序列 attention 按 context 切	KV/attention 通信
Expert Parallel	MoE experts 跨卡分布	token all-to-all

Q. 为什么 3D parallelism 不是“越多并行方式越好”？

A. 每种并行解决不同内存/计算约束，同时引入 collective、bubble 或调度开销。最优组合取决于模型尺寸、序列长度、GPU 拓扑、microbatch 和网络带宽。

8.4 Activation Checkpointing / Recomputation

Q. Activation checkpointing 的本质 trade-off？

A. 前向只保存部分 boundary activations，反向时重算中间激活，用额外 FLOPs 换显存。选择性重算通常比“整层全部重算”更好，需要结合 FlashAttention 等算子的保存策略。

PART 9

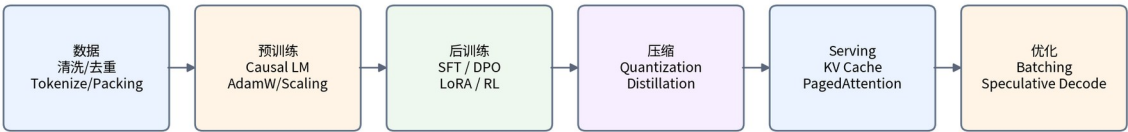
LLM 推理与 Serving

训练看 FLOPs，Serving 更常被 memory bandwidth 与动态调度限制。

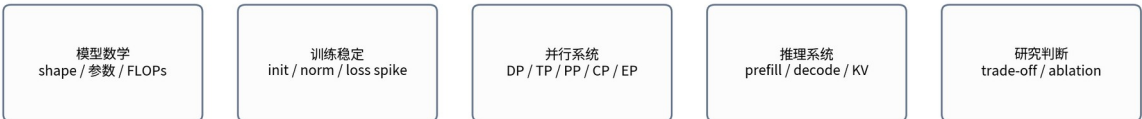
面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

9.1 Prefill vs Decode

Transformer 面试需要贯穿“模型 → 训练 → 系统 → Serving”



高阶面试题通常不是问一个模块的定义，而是要求你解释修改某个模块后，训练稳定性、显存、吞吐和模型质量如何一起变化。



阶段	输入	主要特征	常见瓶颈
Prefill	长 prompt 一次输入	大量 token 并行，GEMM 大	compute / attention IO
Decode	每请求每步 1 token	小 GEMM + 读长历史 KV	memory bandwidth / scheduling

Q. 为什么 decode 往往是 memory-bound?

A. 每步新 token 很少，矩阵计算的 batch/token 维小，但必须读取模型权重和历史 KV；算术强度低，GPU 算力难吃满。Batching 可提高 GEMM 利用率，但受 latency/SLA 限制。

9.2 KV Cache 速算与 Gotchas

$$KV_GiB \approx 2 \cdot B \cdot L \cdot N_layer \cdot N_kv \cdot d_head \cdot bytes / 2^{30}$$

错误	表现	根因
position id 错	cached decode 与 full forward 不一致	RoPE phase / offset 错
beam reorder 漏 KV	生成异常但不一定 crash	beam state 与 cache 未同步
GQA repeat_kv 错	head 内容错位	Q group→KV head 映射错误
window eviction 错	长生成突然退化	sliding window 保留范围/attention sink 错

Q. KV Cache 为什么缓存 K/V 而不是 Q?

A. 历史 token 的 K/V 会被未来所有 query 重用；历史 Q 只用于它自己当时的输出，未来不会再次需要。

9.3 Continuous Batching 与 PagedAttention

30 秒回答

传统静态 batch 要等整批请求完成，长度差异造成 GPU 空闲。Continuous batching 在 token step 粒度动态加入/移除请求。PagedAttention 把 KV cache 类比虚拟内存按 block 管理，减少连续大块预分配导致的 fragmentation，并支持更灵活

的共享/调度。

Q. PagedAttention 会减少 KV Cache 理论字节数吗？

A. 主要减少管理浪费和碎片，不改变单 token K/V 的理论元素数量；真正减少理论 cache 需要 GQA/MQA/MLA、KV quantization 或 eviction/compression。

9.4 Speculative Decoding

- draft model 先提出多个候选 token，target model 一次并行验证。
- 若接受率高，可把多个串行 target decode step 合并成一次 target forward，从而降低 latency。
- 收益取决于 draft 速度、acceptance rate、verification batch、target memory bandwidth；不是所有 workload 都加速。

Q. Speculative decoding 会改变 target model 的输出分布吗？

A. 正确的 speculative sampling/decoding 算法通过接受-拒绝校正确保与 target 分布一致；工程近似方案若省略校正可能改变分布。

9.5 Decoding: Greedy / Temperature / Top-k / Top-p

Q. Greedy decoding 需要显式 softmax 吗？

A. 不需要。 $\text{argmax}(\text{logits}) = \text{argmax}(\text{softmax}(\text{logits}))$ ，因此可直接在 logits 上找最大值。

Q. Temperature 在 softmax 里做什么？

A. $p_i \propto \exp(z_i/T)$ 。T<1 放大 logit gap、分布更尖；T>1 压缩 gap、分布更平。它等价于缩放 logits，不是对概率线性缩放。

Q. Top-p 为什么比 top-k 更自适应？

A. top-k 固定保留 k 个 token；top-p 选择累计概率达到阈值的最小集合，候选数量会随当前分布的不确定性变化。

PART 10

长上下文、效率与量化

“能装下”与“能利用”是两件不同的事。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

10.1 长上下文的三层问题

层次	问题	典型手段
架构长度	position/attention 是否定义	RoPE scaling, relative bias
计算长度	显存与 FLOPs 是否可承受	FlashAttention, CP, sparse/window
有效长度	模型是否真的检索/推理成功	长上下文数据 curriculum, eval, retrieval

Q. 为什么 Needle-in-a-Haystack 通过也不代表长上下文能力强？

A. 它主要测试显式检索；真实任务还包含多跳、聚合、位置偏置、干扰项、长距离 reasoning。应使用多种位置、长度和任务类型评估有效上下文。

10.2 RoPE Context Extension

- Position Interpolation：把更长位置映射回训练过的相位范围。
- NTK/Dynamic/YaRN：对不同频率维进行更细的缩放/插值，兼顾局部分辨率和长距离。
- 扩窗通常需要 continued pretraining / long-context fine-tuning；只改 config 风险很高。

Q. 为什么统一线性缩放所有 RoPE 频率可能伤害短距离能力？

A. 不同频率维负责不同尺度的位置变化；统一压缩会降低高频局部相位分辨率，可能让近距离位置关系变得更难区分。

10.3 Quantization：权重、激活与 KV

对象	目标	典型难点
Weight-only	减少模型权重带宽/显存	outlier channel, calibration
W+A	进一步提速	activation dynamic range
KV Cache	降低长上下文 serving 显存/带宽	误差随长度/层累积
FP8 training	提高训练吞吐	scaling, amax, sensitive ops

Q. 为什么 weight-only quantization 对 decode 特别有价值？

A. decode 常 memory-bound，读取权重是主要带宽开销之一；把权重从 FP16/BF16 降到 8/4bit 可提高有效带宽并扩大 batch。是否加速仍取决于 kernel 和 dequant 开销。

Q. 为什么 Norm/Softmax 往往保留更高精度？

A. 它们包含 reduction、除法/指数等数值敏感操作，并直接控制尺度/概率；量化误差可能被全层放大。实际混合精度 recipe 常把敏感 reduction 放在 FP32/BF16。

PART 11

Debug / 系统设计 / Research Sense

高级面试真正拉开差距的部分。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

11.1 Loss Spike / NaN Debug Playbook

现象	第一检查项	进一步定位
突然 NaN	loss scale / inf grad / batch data	QK logits, norm denominator, exp/log, bad sample
周期性 spike	LR schedule / data shard / checkpoint boundary	特定 domain/token length/worker
后层 RMS 增长	residual updates / norm / init	update/stream ratio, output proj
attention entropy ≈ 0	Q/K norm/logit 太大	QK-Norm, scale, precision
梯度长期被 clip	LR/初始化/数据根因	global vs per-layer norm

- 先确认数据与 loss mask，再看 forward 数值，再看 backward/optimizer；不要一开始就改十个超参。
- 保存 first-bad-step 的 batch、RNG state、model/optimizer state，确保可复现。
- 监控分位数而不是只看全局 mean：某一层/某一 expert/某类 token 的 outlier 常被均值掩盖。

11.2 OOM / Throughput Debug

问题	高概率原因	常用动作
训练 OOM	activation/optimizer/param memory	checkpoint, FSDP/ZeRO, microbatch, seq length
prefill 慢	attention/long prompt	FlashAttention, chunked prefill, CP
decode 慢	KV/weight bandwidth	GQA/MLA, quant, batching
MoE 慢	all-to-all + imbalance	EP mapping, router balance, token dispatch overlap
TP 不扩展	collective latency	节点内 TP, 减 TP degree, overlap

11.3 系统设计题：如何设计一个 70B LLM Serving 服务

- 先明确 SLA：TTFT、TPOT、并发、最大 prompt/context、吞吐优先还是延迟优先。
- 估算权重显存 + KV cache + workspace；决定量化与 TP/PP。
- 将 prefill 与 decode 的瓶颈分开：长 prompt 场景考虑 chunked prefill/prefix cache；高并发 decode 强调 continuous batching。
- 调度层：admission control、KV block manager、preemption、prefix sharing、请求优先级。
- 可靠性：OOM backpressure、timeout、监控、模型 warmup、rolling deployment、cache invalidation。

Q. 为什么系统设计题第一步不是选 vLLM/TensorRT-LLM?

A. 框架只是实现手段。先根据 workload 与 SLA 定义瓶颈，才能决定 batch 策略、KV 容量、量化、并行和调度；否则容易做“技术名词堆砌”。

11.4 Research Sense：如何设计一个公平 Ablation

- 一次只改变一个主要变量；若改变参数量/FLOPs，必须做 parameter-matched 或 compute-matched 对照。
- 固定 tokenizer、数据 order、optimizer、training tokens、eval harness，避免 confounders。
- 至少报告训练曲线、最终指标、吞吐/显存和稳定性；只报一个 benchmark 的最佳点说服力弱。
- 新架构若声称更快，需要区分 theoretical FLOPs、kernel speed、end-to-end throughput 和 serving latency。

Q. 如果 GQA 比 MHA 快且分数相近，能否说 GQA 全面更好？

A. 不能。需说明模型/上下文/硬件/训练预算；MHA 可能在某些任务或模型规模更好。还要确认速度比较是否包含相同 batch、KV cache、kernel 和 TP mapping。

PART 12

终极面试题库与速查表

从 30 秒快问到系统设计与研究追问。

面试目标：定义准确 · 推导完整 · 工程可落地 · 能讲 trade-off

12.1 高频基础题 40 题

Q. Transformer 相比 RNN 最大结构变化？

A. 取消序列递归主干，用 self-attention 实现 token 间交互，并用 FFN 做逐位置非线性计算。

Q. Self-attention 为什么需要位置编码？

A. 内容注意力本身对排列没有绝对顺序感，需要显式位置机制。

Q. Q/K/V 各自职责？

A. Q/K 决定匹配和读取位置，V 是被聚合的信息内容。

Q. 为什么 attention 要 scale？

A. 点积方差随 d_k 增长， $1/\sqrt{d_k}$ 把 logit 尺度稳定在 $O(1)$ 。

Q. Causal mask 做什么？

A. 禁止位置 i 读取未来 $j > i$ 的 token，保证自回归因果性。

Q. MHA 为什么多头？

A. 提供多个并行匹配子空间与不同信息路由模式。

Q. W_O 的作用？

A. 混合各 head 输出并写回 residual hidden dimension。

Q. Pre-Norm 和 Post-Norm 区别？

A. Norm 在 sublayer 前或 residual add 后；Pre-Norm 的 identity gradient path 更直接。

Q. RMSNorm 与 LayerNorm 区别？

A. RMSNorm 不减均值，只按 root-mean-square 缩放，并通常无 beta。

Q. FFN 为什么扩维？

A. 在 channel space 提供更高容量的非线性特征变换。

Q. SwiGLU 为什么三矩阵？

A. gate 与 value 两条上投影逐元素相乘，再 down projection。

Q. Embedding 是什么？

A. 可训练 $V \times d$ 查表，把离散 token id 映射到连续 hidden vector。

Q. Weight tying 是什么？

A. input embedding 与 output unembedding 共享权重。

Q. RoPE 做什么？

A. 按位置旋转 Q/K，使内积包含相对位移相位。

Q. GQA 是什么？

A. 多个 query heads 共享较少 KV heads，降低 KV cache/带宽。

Q. MQA 与 GQA 差异？

A. MQA 只有一个 KV head；GQA 有多个 KV groups。

Q. MLA 的核心？

A. 对 K/V 做低秩 latent joint compression，减少 cache。

Q. FlashAttention 的核心？

A. IO-aware tiling + online softmax，避免 L^2 matrix 在 HBM 往返/物化。

Q. KV Cache 缓存什么？

A. 每层历史 token 的 K/V，用于未来 decode query。

Q. 为什么不缓存 Q?

A. 历史 Q 不会被未来 token 重用。

Q. Prefill 与 decode 区别?

A. prefill 并行处理 prompt; decode 每步新增少量 token 并读取历史 KV。

Q. 为什么 decode memory-bound?

A. 每步算术量小但读权重/KV 多, 算术强度低。

Q. MoE 怎么省 compute?

A. 总专家很多, 但每 token 只激活 Top-K。

Q. MoE 最大工程难点?

A. router balance、capacity、all-to-all 和 expert parallel。

Q. Causal LM objective?

A. 最大化每个 token 在左侧前缀条件下的概率。

Q. SFT 和预训练 loss 区别?

A. 常只对 assistant response token 算监督 loss, 数据分布也变成指令响应。

Q. LoRA 做什么?

A. 冻结基座, 用低秩 BA 表示权重更新。

Q. QLoRA 做什么?

A. 量化冻结基座, 再训练 LoRA adapters。

Q. DDP 的通信?

A. 每卡完整模型, 反向后 all-reduce gradients。

Q. ZeRO-3/FSDP 分片什么?

A. 参数、梯度和 optimizer states 都可分片。

Q. Tensor Parallel 切什么?

A. 切单个大矩阵及其计算。

Q. Pipeline Parallel 切什么?

A. 按层/stage 切模型。

Q. Context Parallel 切什么?

A. 按长序列 context 维分担 attention/activation。

Q. Expert Parallel 切什么?

A. 把 MoE experts 分布到不同 rank。

Q. Activation checkpointing?

A. 少存激活, 反向重算, 用计算换显存。

Q. Top-k 与 top-p 区别?

A. 一个固定候选数, 一个固定累计概率质量。

Q. Temperature 改变什么?

A. 缩放 logits, 控制分布尖锐度/熵。

Q. 长上下文 max length 与有效 length 区别?

A. 能输入不等于能可靠利用所有远距离信息。

Q. Gradient clipping 解决什么?

A. 限制梯度 norm, 缓解单步过大更新; 不能替代根因修复。

Q. Scaling law 的核心?

A. loss 与参数、数据、compute 呈经验幂律，需要联合分配训练预算。

12.2 白板推导题 30 题

Q. 推导 attention logits shape。

A. $X:[B,L,d] \rightarrow Q:[B,H,L,d_h], K:[B,H,L,d_h], QK^T \rightarrow [B,H,L,L]$ 。

Q. 推导 $1/\sqrt{d_k}$ 。

A. 若每维独立 unit variance，则 $q \cdot k$ 方差 $=d_k$ ，除 $\sqrt{d_k}$ 后方差 ≈ 1 。

Q. 推导 cross-entropy gradient。

A. $L = -z_y + \log \sum \exp(z)$ ，对 z_i 求导得 $\text{softmax}_i - 1[i=y]$ 。

Q. 推导传统 FFN 参数。

A. $W1:d \times d_{ff}, W2:d_{ff} \times d$ ，总计 $\approx 2dd_{ff}$ 。

Q. 推导 SwiGLU 参数。

A. gate/up/down 三矩阵，总计 $\approx 3d d_{int}$ 。

Q. 参数匹配 d_{int} 。

A. $3d d_{int} \approx 2d(4d) = 8d^2$ ，所以 $d_{int} \approx 8d/3$ 。

Q. 推导 KV cache。

A. 每层每 token 存 K/V 两份，每份 $N_{kv} \times d_{head}$ 元素，再乘 $B, L, \text{layers}, \text{bytes}$ 。

Q. MHA $\rightarrow$ GQA cache 比例。

A. 其他不变时约按 H_{kv}/H_q 比例缩小。

Q. 为什么 softmax 可减 max?

A. $\exp(z_i - c) / \sum \exp(z_j - c) = \exp(z_i) / \sum \exp(z_j)$ 。

Q. logsumexp 稳定式。

A. $m = \max z$; $LSE = m + \log \sum \exp(z - m)$ 。

Q. 为什么 greedy 不需 softmax?

A. softmax 对 logits 严格单调，argmax 不变。

Q. TP column parallel 的 shape。

A. $W:[d_{in}, d_{out}]$ 按 output dim 切，每 rank 产生 d_{out}/TP 激活。

Q. TP row parallel 的通信。

A. 输入分片，每 rank 计算部分输出贡献，最终 all-reduce sum。

Q. DDP 参数显存。

A. 每卡保存完整 params+grads+optimizer；通信不降低模型副本显存。

Q. ZeRO-1/2/3 分片差异。

A. 依次分片 optimizer、再 gradients、再 parameters。

Q. Attention $O(L^2)$ 来自哪里?

A. 每个 L queries 与 L keys 计算 pairwise score，产生 $L \times L$ interactions。

Q. 为什么 FFN $O(L)$ in sequence?

A. 每 token 独立经过固定矩阵，随 token 数线性增长。

Q. RoPE 相对性。

A. 利用旋转矩阵正交与 $R_m^T R_n = R_{(n-m)}$ 。

Q. Weight tying 参数节省。

A. 不再单独保存第二个 $V \times d$ output matrix, 节省约 Vd 参数。

Q. LoRA 参数量。

A. $A:r \times d_{in}$, $B:d_{out} \times r$, 共 $r(d_{in}+d_{out})$ 。

Q. MoE activated params。

A. 每 token 只计 shared + Top-K expert 路径, 不等于总 expert 参数。

Q. Pipeline bubble 直觉。

A. stage 需等待前后 microbatch, 填充/排空阶段设备利用率不足。

Q. 为何 microbatch 可减 bubble?

A. 更多 microbatches 增加稳态流水占比, 但过多会减 GEMM size/增加调度开销。

Q. Batching 为何提升 decode?

A. 多个请求新 token 合并成较大 GEMM, 提高权重复用和算术强度。

Q. 为什么大词表 LM head 重?

A. 每 token 需 $d \times V$ projection, 参数和计算均线性随 V 增长。

Q. 为什么 sampled softmax 训练可省计算?

A. 只估计目标和部分负类分母; 但改变估计方差/实现复杂度。

Q. 为什么 FP16 softmax 风险大?

A. exp 对大正 logits overflow、小负 logits underflow, 且 reduction 动态范围大。

Q. 为什么 activation checkpoint 增加 FLOPs?

A. 反向前必须重新执行被丢弃的 forward 子图。

Q. 为什么 TP degree 太高反而慢?

A. 单卡 GEMM 变小且 collective 占比/延迟增大。

Q. 为什么长 context decode 每 token 更慢?

A. 每个新 query 要读取/对比更长历史 K/V , 工作量随当前 L 增长。

12.3 工程与 Debug 题 35 题

Q. 训练 loss 突然 NaN, 第一步?

A. 保存 bad batch 与 state, 检查数据/loss mask; 再逐层检查 finite、RMS、QK logits、grad 和 optimizer scale。

Q. full forward 正确但 KV decode 错?

A. 优先检查 position id/RoPE offset、causal mask、cache append、GQA head mapping。

Q. GQA checkpoint 转换后分数掉很多?

A. 检查 KV head averaging/selection 方法、layout、RoPE dims、是否需要短程 uptraining。

Q. 训练一直 gradient clipped?

A. effective LR 过大或某分支尺度异常; clip 只是症状, 检查 per-layer grad/update ratio。

Q. FlashAttention 比 math backend 结果差很多?

A. 应是数值小误差而非语义大差异; 检查 mask/dropout/scale/GQA/layout 和 dtype。

Q. 模型生成不停止?

A. EOS token id、chat template、SFT EOS 数据、stop criterion 或 special-token mapping 错。

Q. 新增 special token 后模型异常?

A. embedding/LM head resize、初始化、tokenizer config、tie weights、继续训练是否一致。

Q. 量化后 perplexity 正常但生成变差?

A. 评估敏感 token/logit ranking、KV quant、outlier layers、sampling 放大误差。

Q. MoE 一些 GPU 长期 100%，另一些低?

A. router load imbalance / expert placement / skewed data, 检查 per-expert tokens。

Q. TP 扩到跨节点吞吐下降?

A. 高频 all-reduce/all-gather 进入慢网络, 尽量节点内 TP、跨节点 DP/PP。

Q. FSDP step time 很慢?

A. 参数 all-gather 太碎、prefetch/overlap 不好、reshard policy 或 microbatch 太小。

Q. PP 有大 bubble?

A. microbatch 数不足、stage compute imbalance、embedding/LM-head stage 过重。

Q. 长 prompt TTFT 很高?

A. prefill 成本大; 考虑 prefix cache、chunked prefill、FlashAttention/CP、prompt compression。

Q. decode TPOT 很高?

A. KV/weight bandwidth、batch 太小、KV cache 太大; 考虑 GQA/quant/batching。

Q. continuous batching 为何仍 OOM?

A. admission control 只看权重没精算 KV worst-case, 或 block fragmentation/workspace 不足。

Q. tokenizer 改动后 loss spike?

A. token distribution/embedding mapping 变化, 需要新 token init、LR 控制和 continued pretraining。

Q. RoPE 扩窗后短任务下降?

A. 频率缩放伤害局部分辨率或训练 curriculum 过度偏长。

Q. attention entropy 极低?

A. logits 尺度过大, 检查 Q/K norm、scale、dtype、mask。

Q. attention entropy 极高?

A. logits 过小或模型未学到 selective routing, 检查 init/scale/QK norms。

Q. embedding rare tokens 学不好?

A. 频率低、gradient starvation; 可调整 vocab、数据采样或初始化/继续训练。

Q. weight tying + 量化有什么坑?

A. input lookup 和 output projection 对精度/布局要求不同, 共享权重让两侧无法独立量化。

Q. SwiGLU FP8 出现 outlier?

A. gate $\times$ up 乘法会放大动态范围, 检查 branch amax/scaling 与敏感层保高精度。

Q. LayerNorm $\rightarrow$ RMSNorm 直接替换可行吗?

A. 不能保证函数等价; 需考虑 mean removal、gamma/beta、checkpoint 迁移与再训练。

Q. 数据 packing 后跨文档泄漏?

A. attention mask/position reset 不正确; 也要决定是否允许跨文档上下文。

Q. SFT loss 很低但聊天差?

A. 模板/数据分布/response mask、过拟合、teacher style 或 eval mismatch。

Q. DPO chosen/rejected 分数拉不开?

A. pair 质量差、差异太弱、reference/ β 设置或 reward signal noisy。

Q. LoRA merge 后结果不同?

A. dtype/rounding、scaling α/r 、target modules、bias 与 adapter order。

Q. Speculative decoding 不加速?

A. draft 太慢、acceptance 低、verification overhead 高或 target 已被大 batch 吃满。

Q. Prefix cache 命中率低?

A. prompt 前缀字节/token 不完全一致, system prompt 动态字段位置不合理。

Q. KV quant 长生成后劣化?

A. 量化误差随层/时间累积, 敏感 head/layer 或 scale granularity 需要调整。

Q. OOM 只发生在偶发 batch?

A. length outlier、MoE routing burst、activation peak、optimizer step workspace。

Q. checkpoint 恢复后 loss 不连续?

A. optimizer/scheduler/RNG/data iterator/grad scaler 未完整恢复。

Q. 训练吞吐慢但 GPU utilization 高?

A. 可能通信/内存 busy 不代表有效 TFLOPs; 看 kernel mix、MFU、network overlap。

Q. MFU 高但 wall-clock 仍差?

A. 模型 FLOPs 定义/overheads、数据输入、checkpoint、eval 时间未计入。

Q. 新架构只在小模型赢, 能否直接 scale?

A. 不能; 需验证 scaling trend、稳定性、kernel efficiency 与 data/compute optimality。

12.4 系统设计 / Research 追问 20 题

Q. 设计 128K context 训练方案。

A. 先 position/attention architecture, 再数据 length curriculum、packing、Flash/CP、显存预算; 用 retrieval+reasoning 多任务评估有效长度。

Q. MHA 改 GQA 如何做公平实验?

A. parameter/compute matched, 固定数据和训练 tokens; 报告 quality、KV bytes、prefill/decode throughput、TP mapping。

Q. 设计 tokenizer vocab size ablation。

A. 保持模型总参数或显式报告差异, 同时比较 fertility、sequence length、LM-head cost、per-language loss。

Q. 设计 MoE expert 数实验。

A. 保持 activated FLOPs/Top-K/训练 tokens, 报告 total params、router balance、all-to-all time、quality。

Q. 如何判断新 Norm 真有效?

A. 不仅看最终 loss; 看最大稳定 LR、warmup 需求、per-layer RMS/grad、深度 scaling 与 end-to-end throughput。

Q. 如何比较 FlashAttention 与 sparse attention?

A. 区分 exact vs approximate; 同硬件同 length/batch 比 latency、memory、quality, 不能只比 asymptotic complexity。

Q. 设计 70B 多机训练并行。

A. 先单卡内存下界, 再选节点内 TP、跨节点 DP/PP; 长序列加 CP, MoE 加 EP; profiling 通信重叠。

Q. 如何定位 data quality 对模型的影响?

A. 固定模型/compute, 构造可控数据 mixture ablation, 训练 proxy model, 再看 held-out loss/target eval/forgetting。

Q. 为什么 benchmark score 不是唯一训练目标?

A. 可能受污染、格式、能力覆盖有限; 还要看 calibration、long-tail、robustness、utility。

Q. 如何设计 KV cache 压缩实验?

A. 固定模型输出质量, 比较 cache bytes/token、decode bandwidth、TPOT、长序列误差累积。

Q. 什么时候选择 MQA 而非 GQA?

A. 极端 serving memory/bandwidth 约束且质量损失可接受；需基于具体 workload 验证。

Q. 什么时候不该用 MoE?

A. 小模型/低并发/网络差/部署简单性优先时，expert communication 与调度可能抵消容量收益。

Q. 如何评估 speculative decoding?

A. 按 workload 报 acceptance length、draft cost、target verify cost、TTFT/TPOT、batch interaction。

Q. 如何判断量化是 memory 还是 compute 优化?

A. 看 kernel 和瓶颈：若 memory-bound，bit reduction 常直接有利；compute-bound 需硬件低比特 tensor core 支持。

Q. 为什么长上下文训练会影响短文本?

A. position distribution、attention pattern、batch token composition 与 optimizer updates 都变了；需要 mixed-length curriculum。

Q. 如何设计 loss spike 自动告警?

A. 监控 loss/grad/RMS/logits/amax 的 EWMA+分位数，触发保存 batch/state 和局部 trace。

Q. 如何研究 head redundancy?

A. pruning/ablation 需要区分 post-hoc 可删与 training-time necessity，并做重新训练/不同 seeds。

Q. 模型参数翻倍，吞吐一定减半吗?

A. 不一定；受 parallelism、GEMM efficiency、memory bandwidth、batch、MoE activation 与通信影响。

Q. 如何解释“质量-速度 trade-off”?

A. 先定义可比单位：参数、activated FLOPs、tokens/s、latency、memory、数据/训练预算，然后谈 Pareto frontier。

Q. 面试遇到陌生架构怎么办?

A. 先定位它改了 Transformer 哪个接口：position、attention score、KV cache、residual、FFN、routing、loss 或 system；再沿 shape/compute/memory/gradient 四条线推导。

12.5 公式速查表

主题	公式 / 速记
Attention	$\text{softmax}((\mathbf{Q}\mathbf{K}^T + \mathbf{M})/\sqrt{d_k})\mathbf{V}$
Softmax stability	$\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} - \max(\mathbf{z}))$
Cross entropy	$L = \log\text{sumexp}(\mathbf{z}) - \mathbf{z}_y$
CE gradient	$\partial L / \partial \mathbf{z} = \mathbf{p} - \text{onehot}(\mathbf{y})$
Dense FFN params	$\approx 2 d d_{\text{ff}}$
SwiGLU params	$\approx 3 d d_{\text{int}}$
Classic matched SwiGLU width	$d_{\text{int}} \approx 8d/3$
Embedding/LM head params	$V \times d$ each; tying removes one copy
KV cache	$2 B L N_{\text{layer}} N_{\text{kv}} d_{\text{head}}$ bytes
LoRA params	$r(d_{\text{in}} + d_{\text{out}})$
RoPE relative property	$\mathbf{R}_m^T \mathbf{R}_n = \mathbf{R}_{(n-m)}$
AdamW	adaptive update + decoupled $\lambda\theta$ decay

12.6 典型模型配置速读法

- 看到 hidden_size d、num_heads H，先算 $d_{\text{head}}=d/H$ ；再看 num_key_value_heads 判定 MHA/GQA/MQA。
- 看到 intermediate_size，判断 dense FFN 还是 SwiGLU；若 SwiGLU，别拿 4d 经验直接比较。
- 看 norm 类型与 placement：LayerNorm/RMSNorm、Pre/Post/Peri；它决定训练稳定性解释框架。
- 看 position：learned absolute、RoPE theta/scale、relative bias；决定 context extension 的方式。
- 看 vocab size 与 tie_word_embeddings；大词表可能显著改变参数分布与 pipeline stage balance。
- 看 MoE：num experts、Top-K、shared experts、capacity/load balancing；总参数与 activated 参数必须分开。

12.7 30 秒回答模板

万能结构

① 一句话定义；② 写核心公式/shape；③ 解释为什么这样设计；④ 给一个关键 trade-off；⑤ 给一个工程 gotcha。这样能把“背概念”升级成“真理解”。

例：为什么用 GQA？

定义：多个 Q heads 共享较少 KV heads。

公式/成本： $KV \text{ cache} \propto N_{kv_heads}$ 。

why：decode 每步需要读取历史 K/V，通常 memory-bandwidth bound。

Trade-off：共享更多可省显存/，但可能牺牲量。

Gotcha：TP world size 与 KV head 不整除，layout/replication 需要特别处理。

参考文献与主线论文

- [1] Vaswani et al. (2017), Attention Is All You Need, arXiv:1706.03762.
- [2] Devlin et al. (2018), BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, arXiv:1810.04805.
- [3] Raffel et al. (2020), Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer, JMLR / arXiv:1910.10683.
- [4] Kaplan et al. (2020), Scaling Laws for Neural Language Models, arXiv:2001.08361.
- [5] Hoffmann et al. (2022), Training Compute-Optimal Large Language Models, arXiv:2203.15556.
- [6] Zhang & Sennrich (2019), Root Mean Square Layer Normalization, arXiv:1910.07467.
- [7] Shazeer (2020), GLU Variants Improve Transformer, arXiv:2002.05202.
- [8] Su et al. (2021), RoFormer: Enhanced Transformer with Rotary Position Embedding, arXiv:2104.09864.
- [9] Shazeer (2019), Fast Transformer Decoding: One Write-Head is All You Need, arXiv:1911.02150.
- [10] Ainslie et al. (2023), GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints, arXiv:2305.13245.
- [11] Dao et al. (2022), FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, arXiv:2205.14135.
- [12] Dao (2023), FlashAttention-2, arXiv:2307.08691; Shah et al. (2024), FlashAttention-3, arXiv:2407.08608.
- [13] Fedus, Zoph & Shazeer (2021/2022), Switch Transformers, arXiv:2101.03961.
- [14] DeepSeek-AI (2024), DeepSeek-V2: MLA + DeepSeekMoE, arXiv:2405.04434.
- [15] DeepSeek-AI (2024/2025), DeepSeek-V3 Technical Report, arXiv:2412.19437.
- [16] Rajbhandari et al. (2020), ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, arXiv:1910.02054.
- [17] Narayanan et al. (2021), Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM, arXiv:2104.04473.
- [18] Kwon et al. (2023), Efficient Memory Management for Large Language Model Serving with PagedAttention, arXiv:2309.06180.
- [19] Leviathan et al. (2023), Fast Inference from Transformers via Speculative Decoding; Chen et al. (2023), Speculative Sampling.
- [20] Hu et al. (2021/2022), LoRA: Low-Rank Adaptation of Large Language Models, arXiv:2106.09685.
- [21] Dettmers et al. (2023), QLoRA: Efficient Finetuning of Quantized LLMs, arXiv:2305.14314.
- [22] He et al. (2015/2016), Deep Residual Learning for Image Recognition, arXiv:1512.03385.
- [23] Xiong et al. (2020), On Layer Normalization in the Transformer Architecture, arXiv:2002.04745.
- [24] Wang et al. (2022), DeepNet: Scaling Transformers to 1,000 Layers, arXiv:2203.00555.
- [25] Press, Smith & Lewis (2021), Train Short, Test Long: ALiBi, arXiv:2108.12409.

- [26] Chen et al. (2023), Extending Context Window of LLMs via Positional Interpolation, arXiv:2306.15595.
- [27] Peng et al. (2023), YaRN: Efficient Context Window Extension of Large Language Models, arXiv:2309.00071.
- [28] Press & Wolf (2016), Using the Output Embedding to Improve Language Models, arXiv:1608.05859.
- [29] Yang et al. (2017), Breaking the Softmax Bottleneck, arXiv:1711.03953.
- [30] 本宝典同时综合了本项目此前生成的 Tokenizer、Token Embedding、Position、QKV、Scaled Attention、MHA、Residual、Normalization、FFN/SwiGLU、LM Head 十个专题报告。

附录：面试前 24 小时 Checklist

- ☐ 能在白板上从 $X:[B,L,d]$ 推导 Q/K/V、attention logits、output shape。
- ☐ 能解释 $1/\sqrt{d_k}$ 、stable softmax、CE gradient。
- ☐ 能手算一组 KV cache 显存，并说明 MHA/GQA/MQA 的比例。
- ☐ 能解释 Pre-Norm、RMSNorm、Residual Stream、SwiGLU 为什么成为现代 LLM 常见组合。
- ☐ 能比较 MHA/GQA/MLA、FlashAttention/PagedAttention 的“优化对象”不同。
- ☐ 能比较 DP/TP/PP/CP/EP 并说明通信拓扑。
- ☐ 能把训练 loss spike 按 data→forward→backward→optimizer 顺序排查。
- ☐ 能把 serving 问题拆成 TTFT(prefill) 与 TPOT(decode)。
- ☐ 能解释 tokenizer/vocab 对 sequence length 与 LM head 的双向成本。
- ☐ 能对一个陌生架构提出 parameter-matched / compute-matched ablation。